


UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI INGEGNERIA
ELETTRICA ELETTRONICA E INFORMATICA

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

# Documentazione finale

Studenti:
Ficarra Filippo 1000034832
Guerrera Mario Anthony 1000035663

Anno Accademico 2025/2026

# Sommario

1. Descrizione dell'applicazione ..... 1
2. Schema architetturale ..... 2
3. Microservizi e relative comunicazioni ..... 4
   3.1 UserManager ..... 4
   3.2 DataCollector ..... 5
   3.3 Nginx ..... 7
   3.4 AlertSystem ..... 8
   3.5 AlertNotifierSystem ..... 9
4. Lista delle API Implementate ..... 11
5. Diagrammi di interazione ..... 13
6. Kubernetes ..... 20
7. Test dell'applicazione e risultati ..... 21
8. Scelte progettuali ..... 29
9. Indice delle figure ..... 32

## 1. Descrizione dell’applicazione

L'applicazione sviluppata è un sistema distribuito basato su un'architettura a microservizi progettata per il monitoraggio in tempo reale dei flussi aerei globali. L'obiettivo principale è fornire un servizio scalabile che raccolga dati da un'API esterna (OpenSky Network) e li elabori in base agli interessi specifici degli utenti iscritti.

Il sistema è stato interamente ingegnerizzato per un ambiente cloud-native, utilizzando Kubernetes (tramite lo strumento Kind) per simulare un cluster multi-nodo locale, garantendo l'orchestrazione e la resilienza dei componenti.

L'architettura si articola sui seguenti pilastri fondamentali:

- **Resilienza con Circuit Breaker**: Per garantire la continuità del servizio anche a fronte di instabilità delle API esterne di OpenSky, è stato implementato il pattern Circuit Breaker. Questo componente monitora le interazioni e interrompe temporaneamente le chiamate in caso di guasti o superamento dei limiti di rate-limiting, prevenendo il degradamento delle prestazioni dell'intero cluster.
- **Architettura Event-Driven (Apache Kafka)**: L'integrazione del message broker Apache Kafka ha permesso di trasformare il sistema in un'architettura basata su eventi (EDA). Kafka funge da spina dorsale asincrona, garantendo un totale disaccoppiamento tra la fase di raccolta dati e quella di elaborazione degli allarmi.
- **Gestione Soglie e Notifiche Asincrone**: Il sistema permette agli utenti di definire soglie personalizzate (high_value e low_value) per il numero di voli in arrivo o partenza da specifici aeroporti. Al superamento di tali soglie, l'Alert System genera eventi consumati dall'Alert Notifier System, che provvede all'invio automatico di notifiche via email.
- **API Gateway (NGINX):** NGINX funge da Single Entry Point del sistema. L'API Gateway centralizza il routing del traffico verso i microservizi interni (User Manager e Data Collector), gestendo la terminazione SSL e semplificando l'interfaccia verso il mondo esterno.
- **White-box Monitoring (Prometheus):** Per garantire l'osservabilità del sistema, è stato implementato un meccanismo di monitoraggio "white-box" integrato nei microservizi. Attraverso l'uso di exporter Prometheus, vengono raccolte ed esposte metriche di tipo Counter e Gauge, permettendo un'analisi precisa delle performance del DB, delle chiamate gRPC e dei carichi di lavoro.

## 2. Schema architetturale

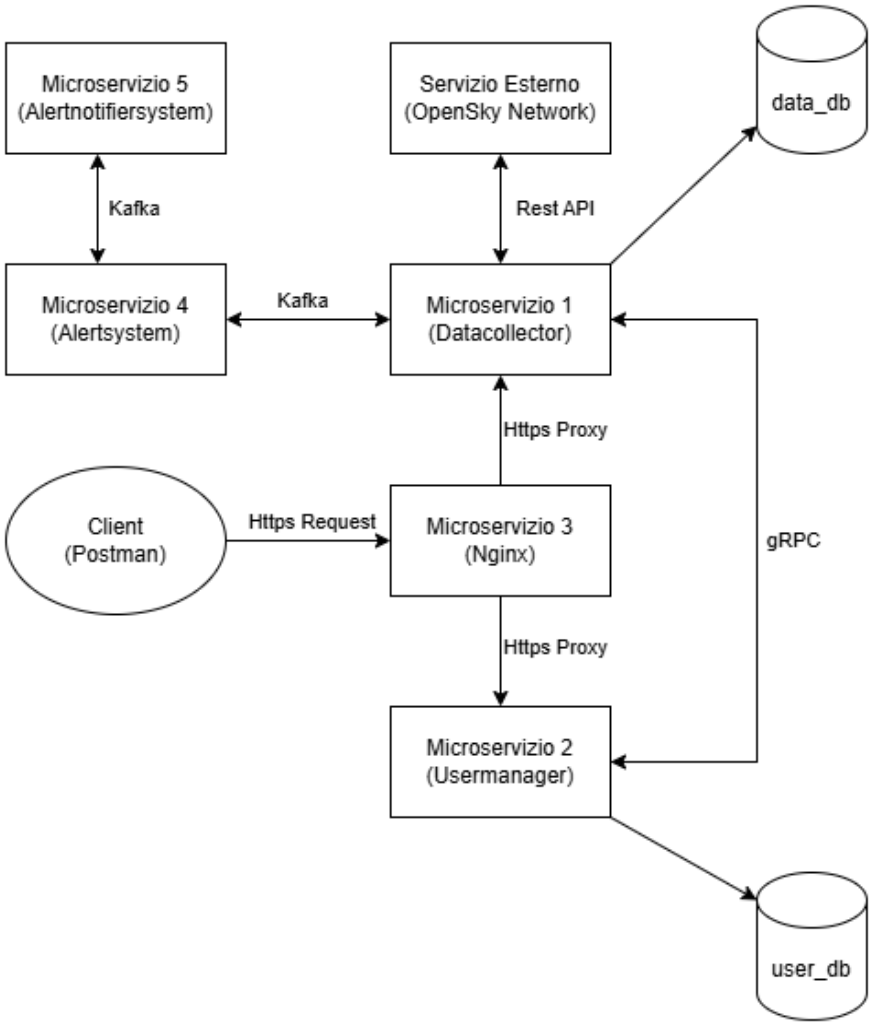
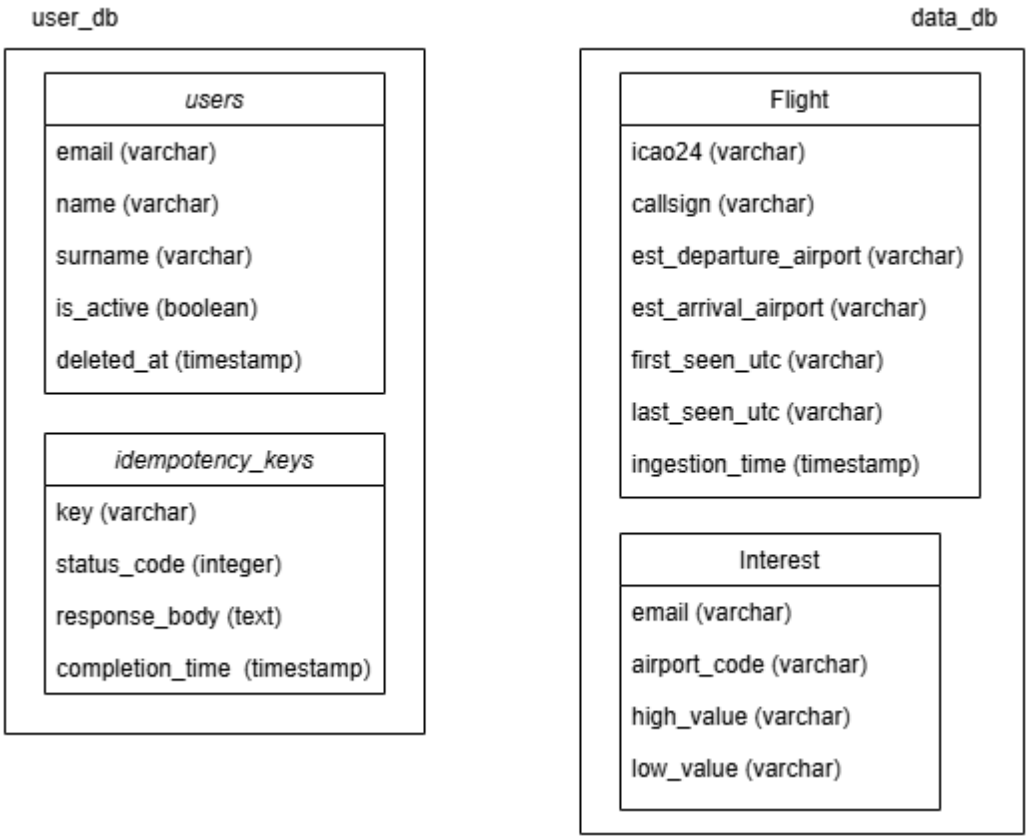



*Figura 1: Schema Architetturale*

L'architettura del sistema si compone di un insieme di moduli interconnessi che collaborano per garantire il flusso dei dati e la resilienza dell'applicazione:

- Punto di Ingresso (API Gateway), rappresentato dal server NGINX, che funge da interfaccia unica verso il mondo esterno (Client/Postman).
- Due microservizi principali, User Manager e Data Collector, responsabili della logica di business.
- Infrastruttura di Messaggistica, tramite l'ausilio di broker Apache Kafka, utilizzato come spina dorsale per la comunicazione asincrona tra i servizi.
- Microservizi di Alerting, ovvero Alert System e Alert Notifier System, dedicati alla valutazione delle soglie e all'invio delle notifiche.
- Servizi esterni: L'integrazione con il servizio OpenSky Network per il reperimento dei dati reali e un server SMTP per la consegna delle email di notifica.

- Stack di Monitoraggio e Orchestrazione: Il sistema include Prometheus per la raccolta delle metriche e l'intero cluster è gestito tramite la piattaforma Kubernetes (Kind).
- Due database relazionali distinti, User DB e Data DB (PostgreSQL), associati, rispettivamente (per garantire la persistenza separata), a User Manager e Data Collector, per la memorizzazione dei profili e dei dati di volo. Nello user_db, oltre alla tabella users, è presente una tabella idempotency_keys, al fine di implementare la politica at-most-once in fase di registrazione dell'utente.



## 3. Microservizi e relative comunicazioni

Di seguito, si elenca una descrizione dei microservizi dell'applicazione e relative comunicazioni. Sono accennate le principali scelte progettuali, alcune affrontate nel dettaglio nella sezione 8 della documentazione.

### 3.1 UserManager

Il microservizio User Manager si occupa della gestione anagrafica degli utenti, interfacciandosi con il database PostgreSQL user_db.

**Scelte Progettuali e Funzionalità:**

- **Affidabilità (At-Most-Once):** Per prevenire duplicazioni dovute a retry di rete, il metodo /add_user implementa un meccanismo di idempotenza lato server. Utilizza un hash SHA-256 dell'email come chiave univoca nella tabella idempotency_keys, garantendo che l'azione avvenga al massimo una volta.
- **Gestione della Cancellazione:** È stata adottata una strategia di Soft Delete (tramite il campo is_active della tabella user) per preservare la consistenza dei dati correlati in altri servizi. La rimozione definitiva avviene in modo asincrono tramite un thread dedicato di Hard Delete, che pulisce periodicamente i record obsoleti in base al PERIODO configurato.
- **Comunicazione Ibrida:** Il servizio espone API REST tramite Flask per le operazioni amministrative e il servizio gRPC UserManagerService (porta 50051) per la validazione sincrona e ad alte prestazioni richiesta dal Data Collector.
- **Osservabilità:** implementa un monitoraggio di tipo **white-box**, esponendo le metriche interne sulla porta dedicata 8000 per facilitare lo scraping da parte di Prometheus:
    - **usermanager_ops_total (Counter)**: Questa metrica viene incrementata ogni volta che viene invocata un'operazione di gestione utente (aggiunta o rimozione), consentendo di monitorare il throughput del servizio e il carico di lavoro amministrativo.
    - **usermanager_db_latency_seconds (Gauge)**: Misura il tempo di esecuzione dell'ultima transazione verso il database PostgreSQL user_db. È uno strumento

diagnostico essenziale per individuare rallentamenti nelle operazioni di persistenza o potenziali colli di bottiglia nel database degli utenti.

- **Label Dinamiche**: Per garantire una granularità avanzata nel monitoraggio all'interno del cluster Kubernetes, le metriche includono le label service e node_name. La label node_name viene popolata dinamicamente tramite variabile d'ambiente, permettendo di correlare le performance del microservizio con lo specifico nodo fisico o virtuale in cui è in esecuzione il Pod.

**Metodi Principali:**

- **CheckUserStatus (gRPC):** Verifica lo stato di un utente (ACTIVE, DELETED, NOT_FOUND) permettendo al sistema di agire in modo coerente durante la raccolta dati.

- **Gestione Threading:** All'avvio, il servizio orchestra quattro thread paralleli per gestire simultaneamente le richieste HTTP, le chiamate gRPC, la pulizia dei dati eliminati e la rotazione delle chiavi di idempotenza.

### 3.2 Data Collector

Il Data Collector è il modulo centrale per l'ingestione e l'elaborazione dei dati di volo. Agisce come un'interfaccia intelligente tra le API esterne di OpenSky Network e il sistema di alerting interno, gestendo al contempo la persistenza degli interessi degli utenti nel database data_db.

**Scelte Progettuali e Resilienza**

- **Isolamento dei Guasti (tramite Circuit Breaker)**: Per garantire la continuità del servizio, è stata adottata una strategia di Circuit Breaker granulare. Oltre a un breaker per l'ottenimento del token OAuth2, è stata implementata una mappa di oggetti CircuitBreaker (airport_breakers), uno per ogni aeroporto monitorato. Questa scelta assicura che un eventuale fallimento o rate-limiting su uno specifico aeroporto non blocchi la raccolta dati per gli altri scali, isolando efficacemente i guasti.

- **Comunicazione Sincrona gRPC**: Il servizio interroga lo User Manager tramite gRPC (tramite la funzione CheckUserStatus) per validare l'esistenza e lo stato dell'utente prima di processare interessi o statistiche. La scelta di gRPC è motivata dalla necessità

di una comunicazione a bassa latenza e fortemente tipizzata, essenziale per non rallentare i job di raccolta ciclici.

- **Architettura Event-Driven (Kafka Producer)**: Il disaccoppiamento tra la raccolta dati e il sistema di notifica è garantito da Apache Kafka. Il Data Collector è configurato come Producer con politiche di affidabilità elevate (acks: all, retries: 3), assicurando che i dati sui voli e le relative soglie utente siano correttamente recapitati al topic to-alert-system.

- **Idempotenza della Persistenza**: Nel salvataggio dei voli viene utilizzato il metodo db.session.merge(). Questa scelta progettuale rende l'ingestione idempotente: se lo stesso volo viene rilevato in finestre temporali sovrapposte, il record esistente viene aggiornato senza generare errori di chiave primaria duplicata.

- **Monitoraggio e Osservabilità (Prometheus):** Il microservizio implementa un monitoraggio di tipo **white-box** esponendo metriche specifiche sulla porta 8000:

    - **datacollector_flights_total (Counter)**: Incrementato ogni volta che nuovi voli vengono salvati con successo nel database, permettendo di monitorare il volume di ingestione dati per aeroporto.
    - **datacollector_grpc_latency_seconds (Gauge)**: Misura il tempo impiegato per la comunicazione gRPC con lo User Manager, fondamentale per identificare colli di bottiglia nella validazione degli utenti.
    - **Label Dinamiche**: Entrambe le metriche includono le label service e node_name (recuperata dall'ambiente Kubernetes tramite la *Downward API*), garantendo la tracciabilità della metrica all'interno del cluster.

**Funzioni Principali**

- **data_collection_job()**: È il core asincrono del microservizio, eseguito da un thread di schedulazione interno (run_scheduler) per non bloccare le API REST. Il job esegue in sequenza:

    1. **verify_users()**: Sincronizzazione degli interessi tramite gRPC, eliminando i dati associati a utenti non più attivi per garantire la coerenza tra i servizi.

2. **Raccolta Protetta**: Recupero dei voli in partenza/arrivo tramite le istanze dedicate del Circuit Breaker.

3. **Pubblicazione**: Assemblaggio del payload JSON (codice aeroporto, totale voli, lista user_thresholds) e invio al broker Kafka (sul topic to-alert-system).

- **publish_flight_count()**: Gestisce la serializzazione del dizionario in array di byte e la scrittura sul topic Kafka, includendo un timestamp per la tracciabilità temporale del messaggio.

- **API REST**: Espone endpoint per la gestione degli interessi (/add_interest, /add_values) e per la fornitura di statistiche avanzate (/average_flights, /flight_duration, ecc), permettendo agli utenti di interrogare i dati storici memorizzati.

**NOTA:** La scelta di inserire la lista user_thresholds direttamente nel messaggio Kafka è dettata dalla volontà di rendere l'Alert System totalmente autonomo. In questo modo, il servizio di alerting non deve interrogare database esterni o comunicare con altri microservizi, ma si limita a consumare i messaggi dal topic in modo puramente reattivo.

### 3.3 Nginx

Il microservizio NGINX funge da API Gateway , stabilendo il Punto di Ingresso Unico e centralizzando la gestione del traffico per le comunicazioni sincrone (HTTP/REST) del sistema.

Il suo funzionamento si basa su tre ruoli interconnessi:

1. Accesso Centralizzato e Reverse Proxy

- **Punto di Contratto (apigate.com):** NGINX risponde all'indirizzo logico apigate.com (server_name), fungendo da interfaccia pubblica e unico indirizzo che i client esterni devono conoscere per accedere al sistema.
- **Reverse Proxy:** NGINX intercetta tutte le richieste indirizzate a apigate.com e le reindirizza (proxy_pass) verso gli indirizzi interni della rete Docker (es. http://datacollector:5000 o http://usermanager:5000).
- **Vantaggi:** Isola i microservizi interni, nascondendo la loro topologia e i loro indirizzi IP/Porte dal mondo esterno, migliorando la sicurezza e la flessibilità architetturale.

2. Instradamento Intelligente (Routing)

- **Logica di Routing**: Utilizza le direttive location per mappare l'URL richiesto al microservizio competente.
- **Gestione degli Endpoint:** Supporta sia le regole statiche (matching diretto del percorso) sia le regole dinamiche (tramite espressioni regolari come location ~* /last_flight/([A-Z0-9]+)$) per instradare le richieste contenenti parametri variabili. La gestione della logica dei parametri è demandata al microservizio di destinazione.

3. Funzionamento nel Contesto Asincrono

- **Mediazione Sincrona:** NGINX gestisce esclusivamente il traffico HTTP bidirezionale (richiesta → inoltro → risposta), mediando le interazioni REST/gRPC.
- **Separazione:** Non è coinvolto nel flusso di eventi Kafka. Questa separazione rafforza il disaccoppiamento: un errore o un *crash* nell'API Gateway (che coinvolge apigate.com) non interrompe la pipeline di notifica asincrona (Alert System/Notifier System).

### 3.4 AlertSystem

Il microservizio opera simultaneamente come Consumer e Producer, fungendo da motore di calcolo della pipeline asincrona. Esso legge dal topic to-alert-system i messaggi prodotti dal Data Collector e implementa la logica di valutazione delle soglie basata su eventi.

**Scelte Progettuali e Funzionalità:**

- **Architettura Stateless e Scalabilità**: Il servizio è progettato per essere totalmente privo di stato (stateless). Non necessita di interrogare database esterni o altri microservizi poiché riceve tutti i dati necessari (voli totali e lista user_thresholds) direttamente dal messaggio Kafka. Questa scelta consente al sistema di scalare orizzontalmente in modo immediato per gestire volumi crescenti di traffico aereo.

- **Logica di Elaborazione (Process and Produce)**: Una volta letto il messaggio e memorizzato nella variabile flight_data, viene invocata la funzione process_and_produce. Questa estrae il codice dell'aeroporto, i voli raccolti e la lista delle soglie utente (variabile thresholds).

- **Valutazione e Filtraggio**: Il servizio itera su ogni elemento della lista verificando se il totale dei voli sia maggiore di high_value o minore di low_value. Qualora i parametri siano nulli, non viene generata alcuna azione; se invece una soglia risulta superata, viene creato un payload specifico. Tale payload contiene il codice aeroportuale, la

condizione verificata (SUPERATA_SOGLIA_ALTA o SUPERATA_SOGLIA_BASSA), il valore della soglia e l'email dell'utente interessato.

- **Serializzazione e Trasmissione**: Il dizionario risultante viene convertito in una stringa JSON e successivamente trasformato in un array di byte (bytes), formato idoneo alla trasmissione su rete tramite Kafka. Infine, il messaggio viene scritto sul topic to-notifier per essere preso in carico dall AlertNotifierSystem.

**NOTA:** L'intero processo rafforza il disaccoppiamento del sistema. Grazie alla lista user_thresholds passata nel messaggio, l'Alert System è totalmente autonomo e non subisce rallentamenti o guasti derivanti dall'indisponibilità dei database o dello User Manager.

### 3.5 AlertNotifierSystem

L'AlertNotifierSystem funge da Consumer finale nella pipeline di notifica asincrona. Il suo compito principale è ricevere gli eventi di allerta dal topic to-notifier (prodotti dall'AlertSystem) e agire come gestore delle notifiche, simulando l'invio di posta elettronica.

**Scelte Progettuali e Resilienza:**

- **Affidabilità At-Least-Once**: Il servizio garantisce che nessuna notifica vada persa implementando una gestione oculata dei commit di Kafka. La scelta di committare l'offset solo dopo il successo dell'invio (o in caso di errore permanente) assicura che, in caso di crash improvviso, il messaggio venga riletto e l'invio ritentato.
- **Integrazione Protocollo SMTP**: La scelta di utilizzare la libreria standard Python smtplib e il protocollo SMTP è motivata dalla volontà di fornire una simulazione realistica in un ambiente di microservizi. L'uso di variabili d'ambiente (SMTP_SERVER, SMTP_PORT) permette di configurare dinamicamente il server di posta esterno senza modificare il codice.

**Funzionamento e Logica Interna**: Il servizio opera in un ciclo infinito (while True), leggendo passivamente i messaggi ed estraendo dal payload JSON le informazioni essenziali (email, aeroporto, condizione e soglia). Si basa su due funzioni fondamentali:

1. **send_notification**: Riceve i dati del messaggio e stabilisce la connessione SMTP (autenticazione e messa in sicurezza del canale). Gestisce la resilienza in base all'esito:
    - Successo / Errore Permanente: Se l'invio riesce o l'indirizzo è inesistente, viene eseguito il consumer.commit per evitare rilavorazioni.
    - Errore Temporaneo: In caso di timeout o disconnessione, il messaggio non viene committato, garantendo che la notifica venga ritentata al ciclo successivo.
2. **format_email_body**: Costruisce l'oggetto e il corpo dell'email in modo dinamico, differenziando il contenuto in base alla condizione rilevata (SUPERATA_SOGLIA_ALTA o SUPERATA_SOGLIA_BASSA).

4. Lista delle API Implementate

| **Nome API** | **Obiettivo** | **Parametri forniti** | **Parametri ritornati** |
|---|---|---|---|
| /add_user | Registra un nuovo utente | email(string)<br>name(string)<br>surname(string) | message(json) |
| /delete_user | Elimina un utente registrato (soft delete) | email(string) | message(json) |
| /add_interest | Aggiunge uno o più aeroporti di interesse per un utente registrato | email(string)<br>airport(List) | message(json) |
| /flights | Recupera i voli associati agli aeroporti di interesse di un utente registrato | email(string) | Per ogni volo:<br>icao_24(string)<br>callsign(string)<br>est_departure_airport(string)<br>est_arrival_airport (string)<br>first_seen_utc (string)<br>last_seen_utc (string)<br>ingestion_time(timestamp) |
| /interest | Recupera gli aeroporti di interesse degli utenti registrati | email(string) | email (string)<br>airport_code (string) (per ogni aereoporto d'interesse dell'utente) |
| /last_flight/<airport_code> | Recupera l'ultimo volo in partenza e in arrivo da un dato aeroporto | airport_code(string) | icao24_departure (string)<br>callsign_departure (string)<br>est_departure_airport_departure (string)<br>est_arrival_airport_departure (string)<br>first_seen_utc_departure (string)<br>last_seen_utc_departure (string) |

| | | | |
|---|---|---|---|
| | | | icao24_arrival (string)<br>callsign_arrival (string)<br>est_departure_airport_arrival (string)<br>est_arrival_airport_arrival (string) first_seen_utc_arrival (string) last_seen_utc_arrival (string) |
| /average_flights/ <airport_code> | Recupera la media degli ultimi X giorni (7 di default) sul numero di voli in partenza e in arrivo da un dato aeroporto | airport_code(string)<br>days (int) | aeroporto_icao (string)<br>periodo_giorni (int)<br>voli_totali (int)<br>media_giornaliera (double) |
| /flights_by_period | Recupera i voli svolti in un determinato intervallo temporale | start_date(string)<br>end_date(string) | start_date_str (string)<br>end_date_str (string)<br>aeroporto (string)<br>voli_totali (int) |
| /flight_duration/ <airport_code> | Recupera il volo più lungo e più breve per un dato aeroporto (che sia l'aeroporto di partenza o arrivo) | airport_code(string) | Per il volo più breve e il volo più lungo:<br>icao24 (string)<br>callsign (string)<br>durata_secondi (int)<br>durata_formattata (strinig) |
| /add_values | Permette di aggiungere uno o entrambi i parametri high_value e low_value | add_values(email, aiport_code, high_value, low_value) | json contenente una stringa di modifica confermata, altrimenti una stringa di errore. |

**NOTA**: Si è deciso di implementare, oltre alle API richieste, due API aggiuntive (descritte all'interno della tabella). Si tratta di: /flights_by_period e /flight_duration/<airport_code>.

## 5. Diagrammi di interazione

Di seguito si riportano alcuni diagrammi di interazione delle funzioni principali dell'applicazione. Si è scelto di riportare le operazioni più significative, e non tutte, per evitare di appesantire troppo la documentazione.

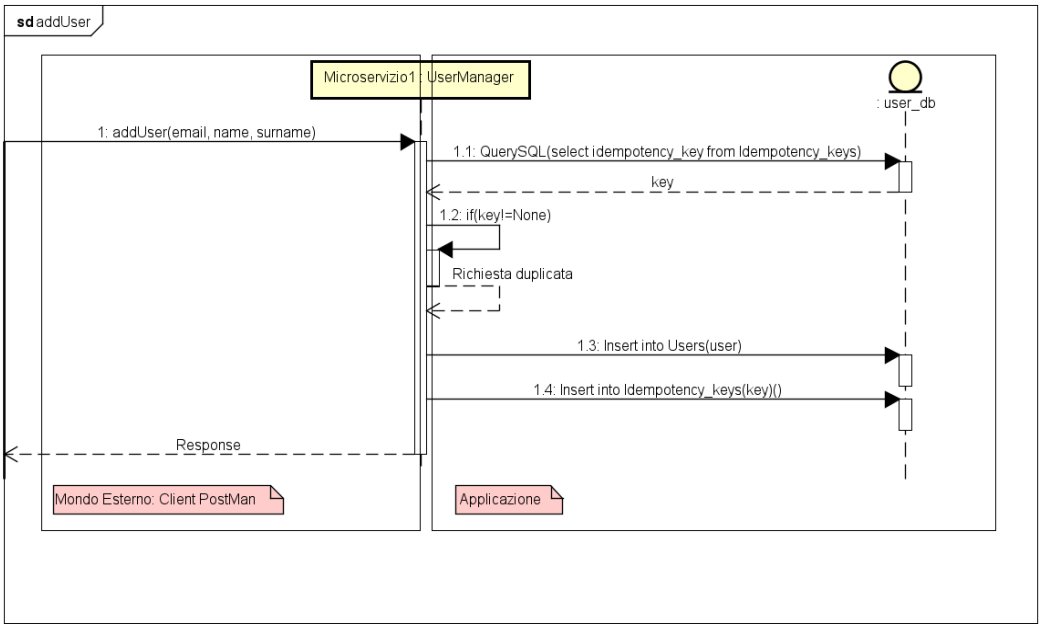



*Figura 2: add_user*

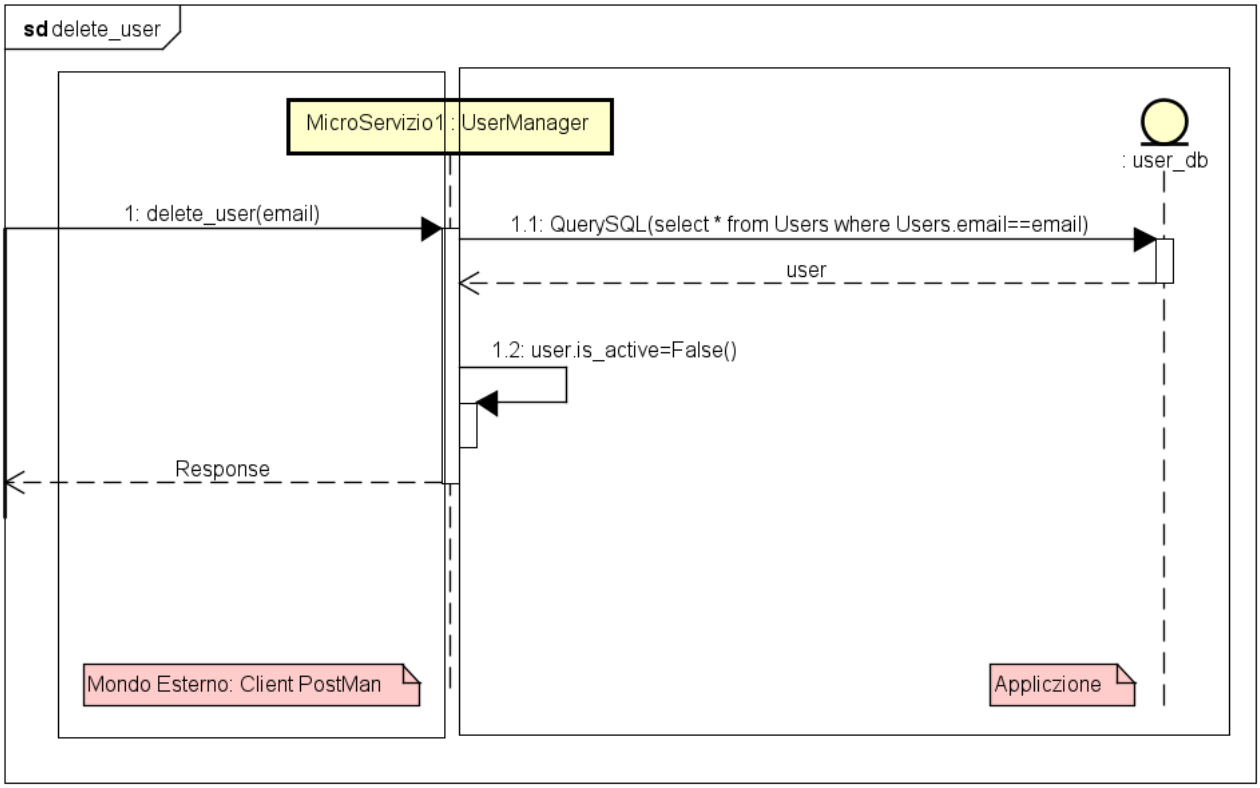



*Figura 3: delete_user*

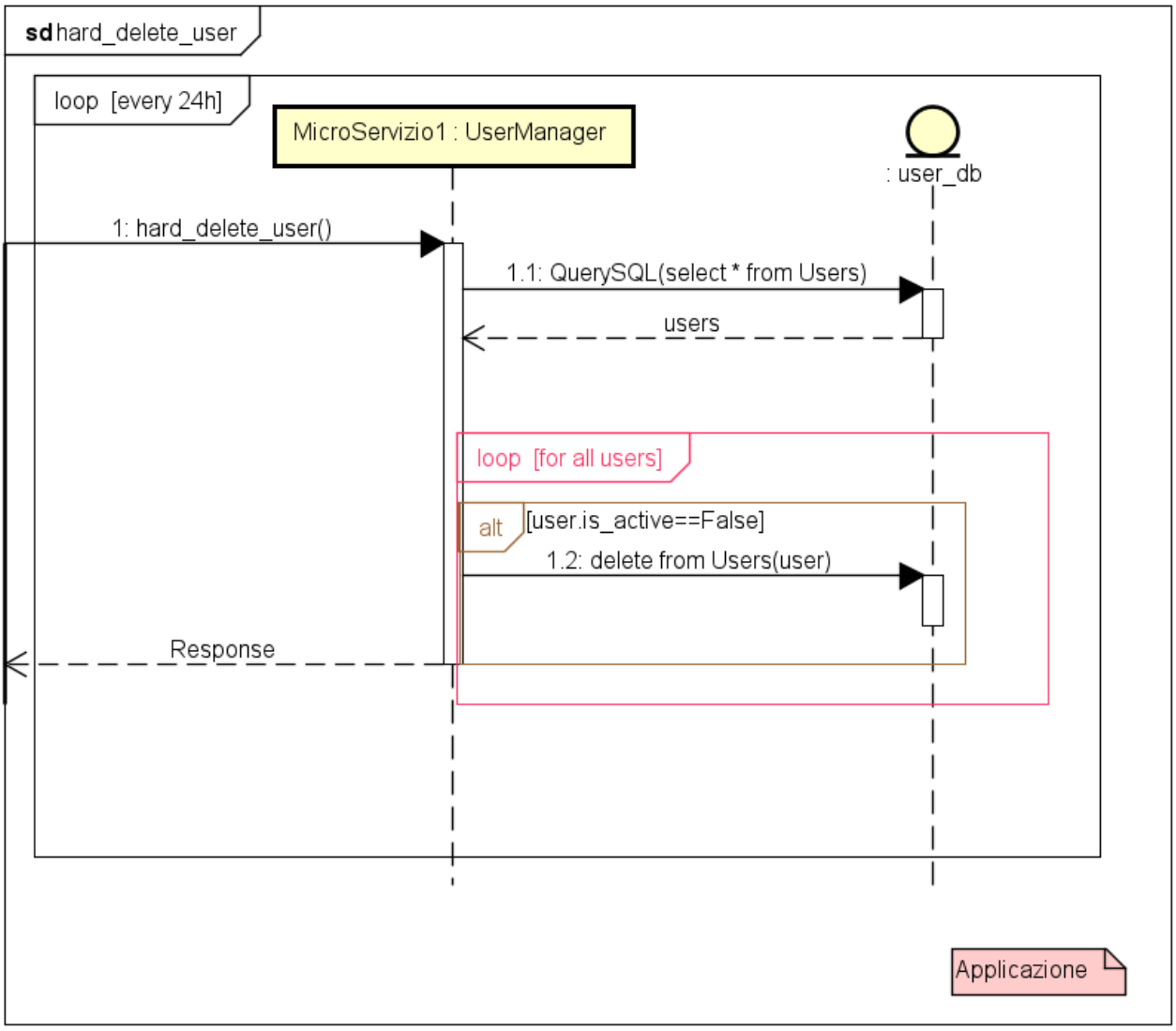



*Figura 4: hard_delete_user*

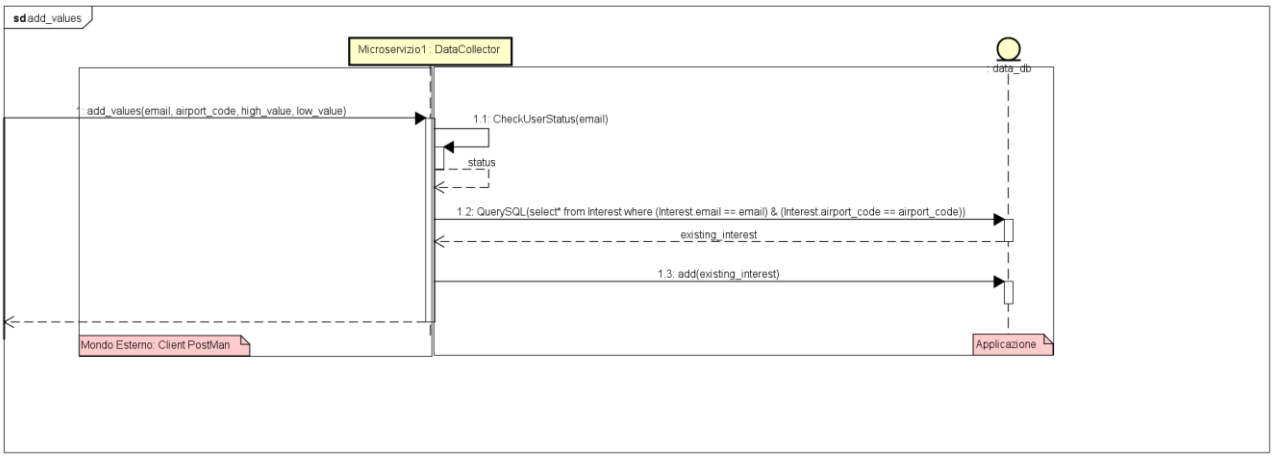



*Figura 5: add_values*

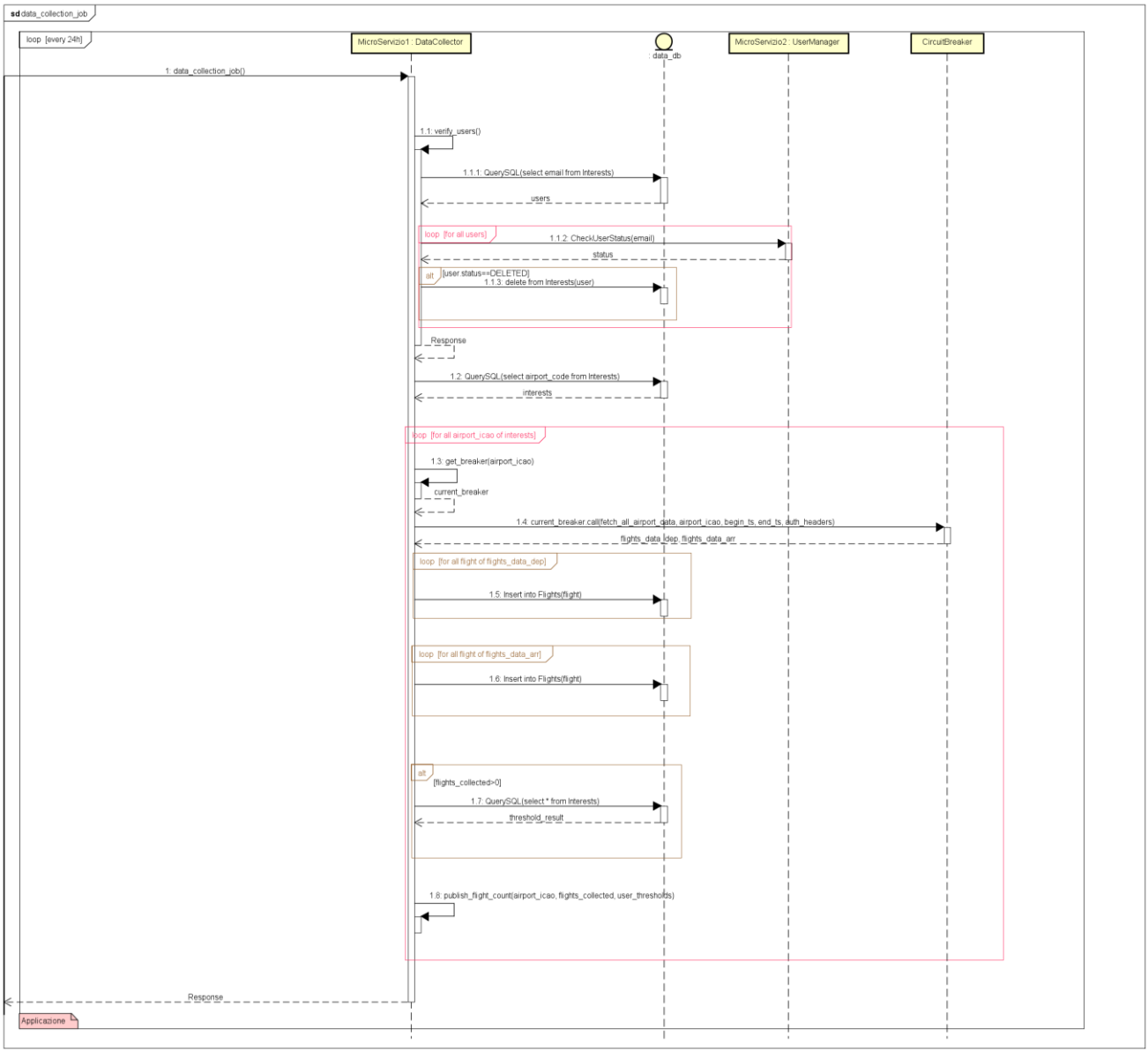



*Figura 6: data_collection_job*

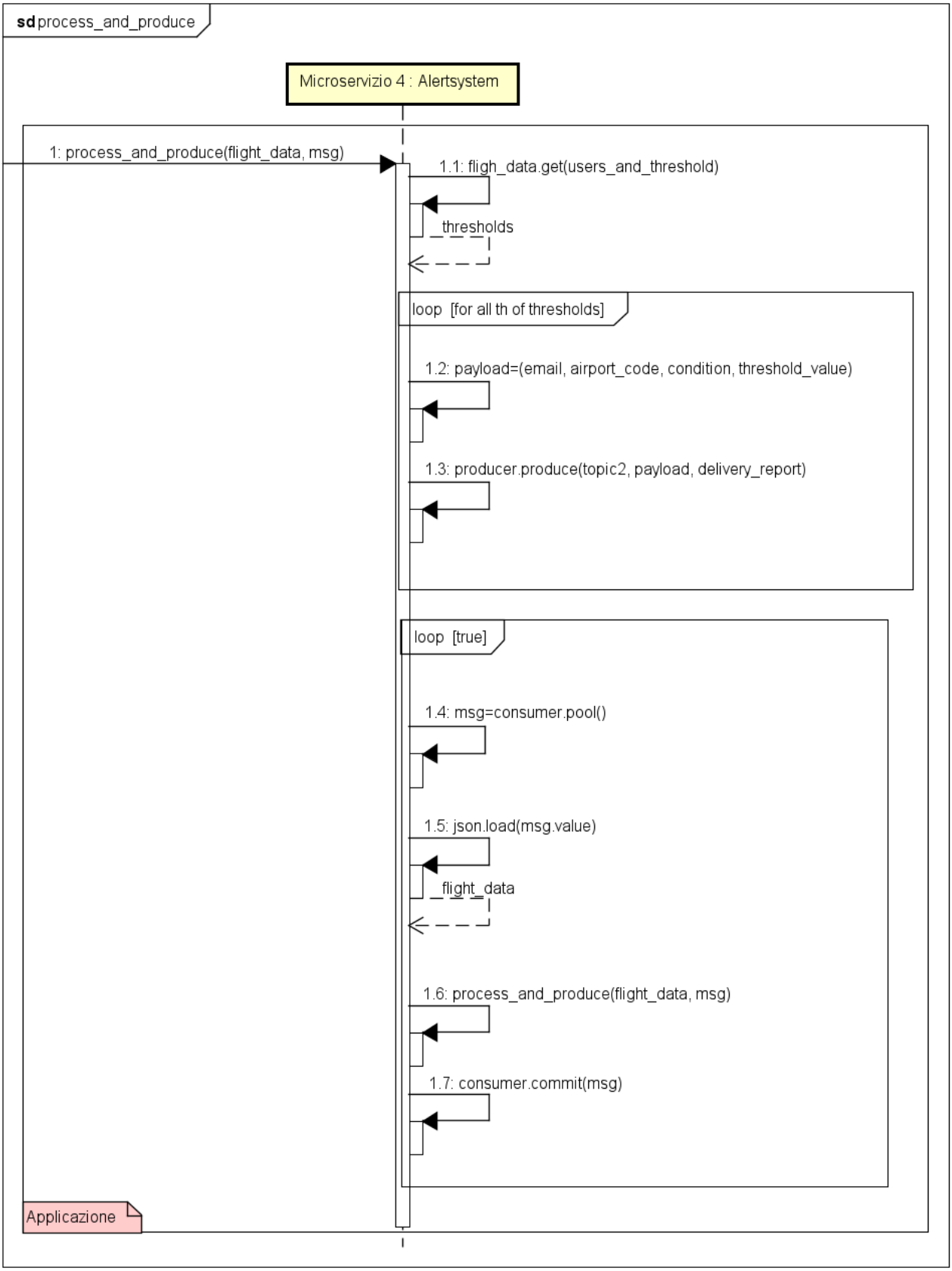



*Figura 7: process_and_produce*

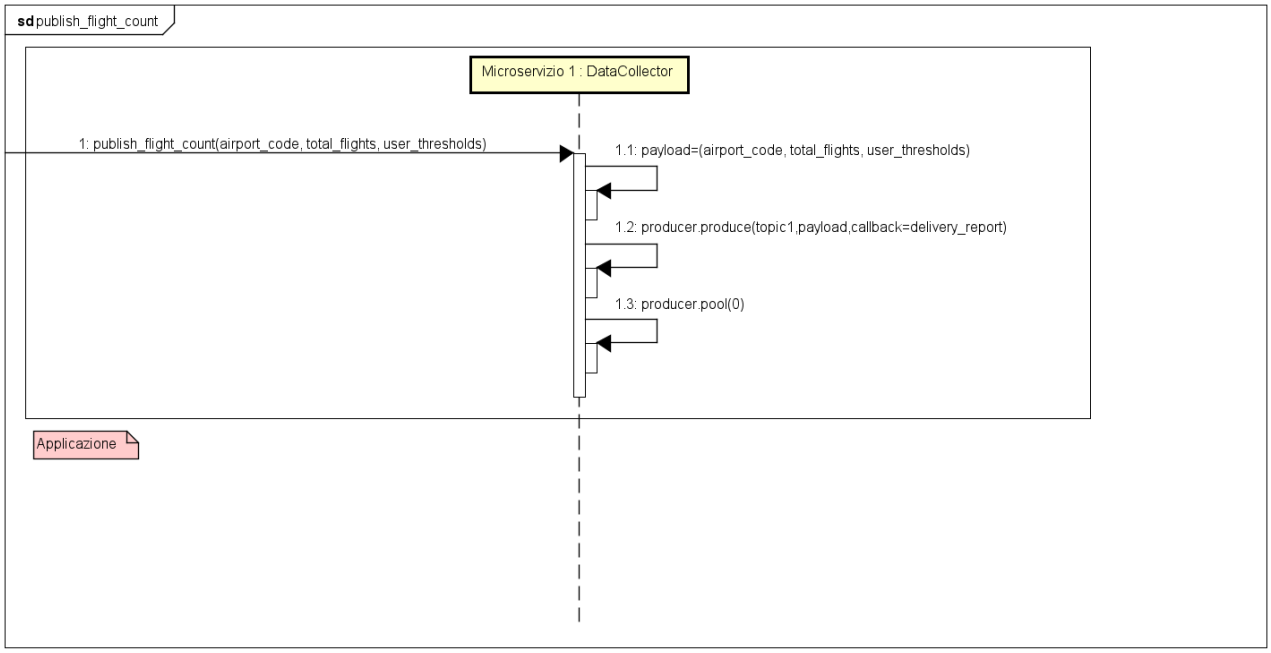



*Figura 8: publish_flight_count*

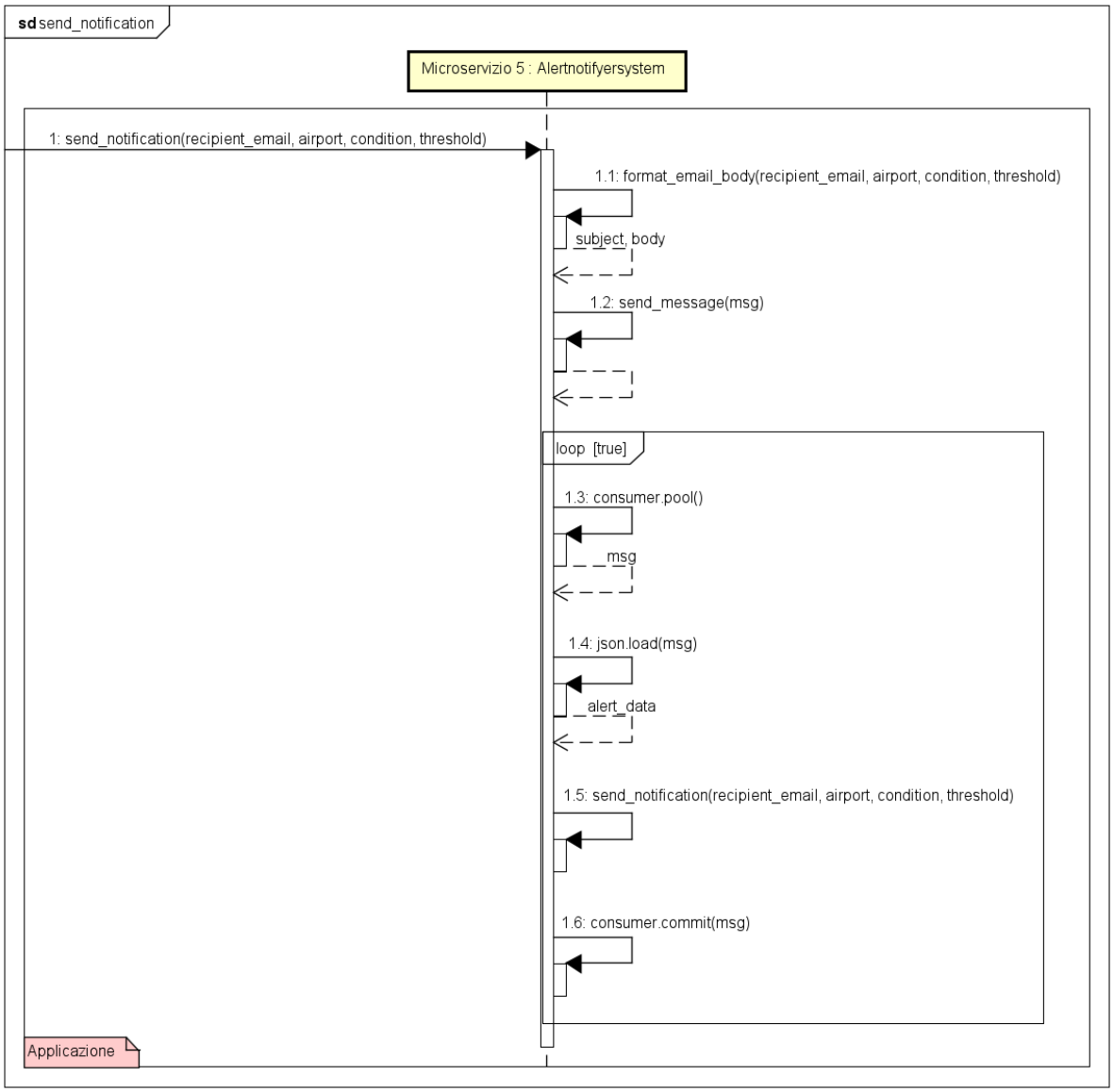



*Figura 9: send_notification*

## 6. Kubernetes

Il progetto ha previsto inizialmente l'orchestrazione tramite Docker Compose, un metodo di configurazione dichiarativo, semplice, ma con il grosso vincolo del single-host; cioè progettato per funzionare su singola macchina.

Il deployment su Kubernetes permette di sviluppare l'applicazione per un ambiente cloud-native. Grazie ai manifest contenuti nella directory k8s si danno le istruzioni al cluster su come gestire i pods. Inizialmente è stato creato un cluster kind che viene personalizzato grazie al manifest kind-config.yaml in cui mappiamo la porta 30443 del container con la porta 443 dell'host. Successivamente viene applicato al cluster il file kustomization.yaml che raggruppa gli altri file yaml e permette di applicarli in ordine con un solo comando. I manifest kubernetes compresi nel file kustomization sono:

- namespace.yaml: crea il namespace 'homework' come spazio di risorse per isolare l'ambiente.
- configmap-postgres.yaml: inizializza le tabelle di user-db e data-db.
- postgres.yaml: crea le due istanze di database PostgreSQL, user-db e data-db, il loro indirizzo utilizzato dagli altri microservizi, imposta le credenziali e avvia i file di configurazione contenuti nella configmap-postgres.yaml.
- kafka.yaml: crea il servizio kafka-hs per gestire i nomi DNS, il servizio StatefulSet kafka-0 (anche in caso di riavvio il nome del pod sarà sempre kafka-0).
- app-service.yaml: lancia i microservizi datacollector e usermanager definendo le relative porte e variabili d'ambiente.
- alerts.yaml: lancia i microservizi alertsystem e alertnotifiersystem definendo le relative porte e variabili d'ambiente.
- prometheus.yaml: lancia e configura il microservizio prometheus per ottenere i labels definiti nei microservizi.
- nginx.yaml: lancia e configura Nginx per utilizzare l'apigateway.

L' applicazione, grazie al deployment su kubernetes, permette la scalabilità, cioè replicare i microservizi per rispondere ad una maggiore richiesta, permette la configurazione delle variabili segrete grazie al file secrets.yaml.

# 7. Test dell'applicazione e risultati

I passaggi da eseguire per l'esecuzione dell'applicazione sono riportati nel file "README.md" all'interno del progetto.

Si noti come, per l'esecuzione con Kubernetes, è sufficiente eseguire lo script exe.ps1. La scelta di inserire questo file è dovuta alla volontà di semplificare i passaggi da effettuare per l'esecuzione dell'applicazione. Questo script automatizza non solo il kubectl apply, ma anche la gestione dei certificati e delle variabili segrete, come la password per app, o la password di OpenSky.

Per evitare di sovraccaricare la documentazione, e di inserire immagini poco leggibili, si riportano alcuni screen relativi alle API testate all'interno della directory "api_screen" del progetto. Per semplicità, si è scelto di riportare solo i casi di successo, anche se sono stati testati pure i casi di fallimento, correttamente funzionanti. In ogni caso, in ogni screen è riportato l'URL testato e i parametri passati (come Json o all'interno dell'URL, a seconda della richiesta), in modo che, chiunque visualizzi la relazione e gli screen, sia in grado di testare il funzionamento.

Si noti come, nelle seguenti immagini, gli URL usati per invocare le opportune funzioni, rispettano il funzionamento di NGINX, e di come viene utilizzato "apigate.com" al posto dell'indirizzo del microservizio.

## 7.1 Analisi dei risultati dei test sulle API

Come si evince dall'immagine "add_user.png", l'utente viene aggiunto correttamente al database, e, grazie all'implementazione della politica "at-most-once", viene individuata una richiesta duplicata, come si vede dall'immagine "add_user (politica at-most-once).png". L'immagine "user.png" conferma il funzionamento dell'API, con cui è possibile vedere l'elenco di utenti registrati.

Vedendo lo screen "delete_user.png", testato su un utente prima aggiunto e poi eliminato, la soft delete viene eseguita correttamente.

L'API "/add_interest" viene testata per due aeroporti differenti. Nell'immagine "add_interest.png" si utilizza l'aeroporto LIMC, e si è deciso volutamente di non riportare high_value e low_value in modo da dimostrare che possono anche essere omessi e aggiunti

successivamente. Invece, nello screen “add_interest (2).png” è stato aggiunto l’aeroporto LICC e si è deciso di riportare solo il low_value, aggiungendo l’high value successivamente.

Gli screen sui test dell’API /add_values dimostrano che è possibile sia aggiungere i valori soglia per un interesse (se è presente nella tabella ovviamente) sia modificare i valori già presenti:

- Come si vede dallo screen “add_values.png”, sono stati aggiunti i due valori soglia per l’aeroporto LIMC.
- Come si vede nello screen “add_values (2).png”, è stato aggiungo l’high_value per l’aeroporto LICC.
- Nello screen “add_values(3).png” è stato modificato il low_value per l’aeroporto LICC.
- Nell’immagine “add_values(4).png” sono state modificate, dopo le prime esecuzioni del data_collection_job(), le due soglie, in modo da testare il funzionamento di kafka.

Dall’immagine “interest.png” si evince il funzionamento della rispettiva API, selezionando correttamente gli interessi relativi alla mail fornita.

Le API di elaborazione sui dati relativi ai voli raccolti funzionano correttamente:

- Dall’immagine “flights.png” si vede che, passando come parametri (in un JSON) la mail dell’utente e l’aeroporto di interesse, vengono restituiti tutti i voli associati a quell’aeroporto.
- L’immagine “last_flight.png” dimostra il funzionamento della rispettiva API, in cui, passando all’interno dell’URL l’aeroporto di interesse (nell’esempio LIMC), viene restituito l’ultimo volo, in partenza e in arrivo, da quell’aeroporto.
- L’api /average_flights è stata testata sia senza specificare il numero di giorni, come si vede dell’immagine “average_flights (days default).png”, sia passando un parametro days=25, come si evince nell’immagine “average_flights.png”. In entrambi i casi, funziona correttamente.

Sono state testate anche le due api aggiuntive:

- Nello screen “flights_by_period.png” si evince come, dopo aver passato le due date (start_date e end_date) come parametri, l’API consente di recuperare tutti i voli tracciati in quell’intervallo temporale.

- L'API /flight_duration, come si evince dall'immagine "flight_duration.png", funziona correttamente. Passando l'aeroporto di interesse all'interno dell'URL (nell'esempio è LIMC), vengono recuperati il volo più lungo e più breve in partenza o in arrivo dall'aeroporto specificato. Bisogna evidenziare come, nell'esempio analizzato, l'aeroporto più breve ha una durata di 106 secondi, che chiaramente è troppo breve per un volo. Tuttavia, si potrebbe trattare di un errore del servizio esterno che fornisce informazioni sui voli, mentre la logica della funzione rimane corretta (in quanto seleziona il minimo delle durate fornite).

7.2 Analisi dei risultati sul test del data_collection_job() e del circuit Breaker

```
DEBUG API: Chiamata Partenze per LICC
Crediti OpenSky Rimanenti: 850
DEBUG API: Chiamata Arrivi per LICC
Crediti OpenSky Rimanenti: 820
Errore non gestito durante la raccolta dati per LICC: 404 Client Error:  for url: https://opensky-network.org/api/fli
Client: invio la richiesta con email: filip02.ficarra@gmail.com
Ricevuta risposta: 1
DEBUG: Breaker ID 136293475683984 sta gestendo la chiamata
DEBUG API: Chiamata Partenze per LIMC
Crediti OpenSky Rimanenti: 790
DEBUG API: Chiamata Arrivi per LIMC
Crediti OpenSky Rimanenti: 760
Salvati 42 voli totali per LIMC.
[KAFKA PRODUCER] Inviati dati su LIMC a to-alert-system.
DEBUG: Breaker ID 136293475250640 sta gestendo la chiamata
[LICC] Circuit Breaker APERTO. Saltata la raccolta per questo aeroporto.
Client: invio la richiesta con email: filip02.ficarra@gmail.com
Ricevuta risposta: 1
DEBUG: Breaker ID 136293475683984 sta gestendo la chiamata
DEBUG API: Chiamata Partenze per LIMC
Crediti OpenSky Rimanenti: 730
DEBUG API: Chiamata Arrivi per LIMC
Crediti OpenSky Rimanenti: 700
Salvati 42 voli totali per LIMC.
[KAFKA PRODUCER] Inviati dati su LIMC a to-alert-system.
```

Il seguente screen dimostra come, se presenti, vengono raccolti correttamente i voli per gli aeroporti di interesse. I voli vengono raccolti ogni 20 secondi, il valore a cui è stata settata la variabile d'ambiente PERIODO.

Per l'aeroporto LIMC vengono raccolti correttamente i voli (totali di partenze e arrivi) e scritti sul topic "to-alert-system". Per l'aeroporto LICC non sono tracciati voli dal servizio esterno, e di conseguenza, si ha un errore 404. Dopo 5 fallimenti della richiesta, anche se non sono riportati tutti nello screen per motivi di lunghezza e leggibilità, si attiva il circuit breaker per LICC, mentre, dal momento che è stata utilizzata una mappa con circuit breaker diversi per aeroporto, sarà possibile continuare a monitorare e raccogliere i voli di LIMC. Infatti, come si vede dallo screen, ogni circuit breaker ha un ID diverso.

```
DEBUG: Breaker ID 136293475250640 sta gestendo la chiamata
DEBUG: Breaker 136293475250640 passa a HALF_OPEN
DEBUG API: Chiamata Partenze per LICC
Crediti OpenSky Rimanenti: 550
DEBUG API: Chiamata Arrivi per LICC
Crediti OpenSky Rimanenti: 520
Errore non gestito durante la raccolta dati per LICC: 404 Client Error:  for url: https://opensky-network.org/api/flights/ar
Client: invio la richiesta con email: filip02.ficarra@gmail.com
Ricevuta risposta: 1
DEBUG: Breaker ID 136293475683984 sta gestendo la chiamata
DEBUG API: Chiamata Partenze per LIMC
Crediti OpenSky Rimanenti: 490
DEBUG API: Chiamata Arrivi per LIMC
Crediti OpenSky Rimanenti: 460
Salvati 43 voli totali per LIMC.
[KAFKA PRODUCER] Inviati dati su LIMC a to-alert-system.
DEBUG: Breaker ID 136293475250640 sta gestendo la chiamata
[LICC] Circuit Breaker APERTO. Saltata la raccolta per questo aeroporto.
```

Dopo un recovery_time, impostato a 120 secondi, il circuit breaker relativo a LICC passa correttamente in stato di half_open e ritenta la richiesta. Poiché questa fallisce nuovamente, il circuit breaker ritorna in stato di open, il tutto in maniera indipendente rispetto al circuit breaker di LIMC, che ha il proprio.

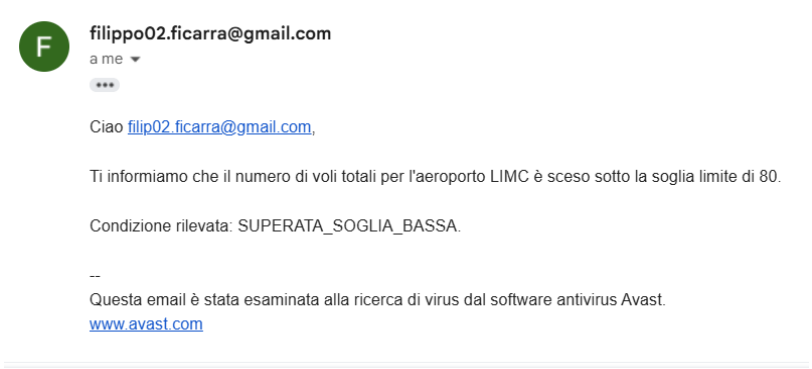
### 7.3 Analisi dei risultati sui test di AlertSystem e AlertNotifierSystem

I voli raccolti dal data collector vengono scritti correttamente nel topic “to-alert-system”, e sono elaborati correttamente dall’ AlertSystem

```
[COMMIT SUCCESS] Messaggio 7 elaborato e offset committato.
[RECEIVED] Dati volo da DC: LIMC (34 voli)
Check and notify con 1 profili
[COMMIT SUCCESS] Messaggio 8 elaborato e offset committato.
[RECEIVED] Dati volo da DC: LIMC (35 voli)
Check and notify con 1 profili
[COMMIT SUCCESS] Messaggio 9 elaborato e offset committato.
[RECEIVED] Dati volo da DC: LIMC (35 voli)
Check and notify con 1 profili
[ALERT PRODUCER] Notifica inviata per filip02.ficarra@gmail.com: soglia SUPERATA_SOGLIA_BASSA su LIMC
[COMMIT SUCCESS] Messaggio 10 elaborato e offset committato.
[RECEIVED] Dati volo da DC: LIMC (35 voli)
Check and notify con 1 profili
[ALERT PRODUCER] Notifica inviata per filip02.ficarra@gmail.com: soglia SUPERATA_SOGLIA_BASSA su LIMC
[COMMIT SUCCESS] Messaggio 11 elaborato e offset committato.
[RECEIVED] Dati volo da DC: LIMC (35 voli)
Check and notify con 1 profili
[ALERT PRODUCER] Notifica inviata per filip02.ficarra@gmail.com: soglia SUPERATA_SOGLIA_BASSA su LIMC
[COMMIT SUCCESS] Messaggio 12 elaborato e offset committato.
[RECEIVED] Dati volo da DC: LIMC (35 voli)
Check and notify con 1 profili
[ALERT PRODUCER] Notifica inviata per filip02.ficarra@gmail.com: soglia SUPERATA_SOGLIA_BASSA su LIMC
```

Si noti come, prima che venissero modificati i valori soglia dell’aeroporto LIMC con add_values, non viene scritto alcun messaggio sul topic “to-notifier”, in quanto il numero di voli era compreso tra le soglie. Se una delle soglie viene superata, l’alertSystem scrive correttamente nel topic “to-notifier”, e l’AlertNotifierSystem legge correttamente dal topic e invia la mail di notifica.

```
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
[EMAIL SUCCESS] Notifica inviata a filip02.ficarra@gmail.com.
```



**filippo02.ficarra@gmail.com**
a me

Ciao filip02.ficarra@gmail.com,

Ti informiamo che il numero di voli totali per l'aeroporto LIMC è sceso sotto la soglia limite di 80.

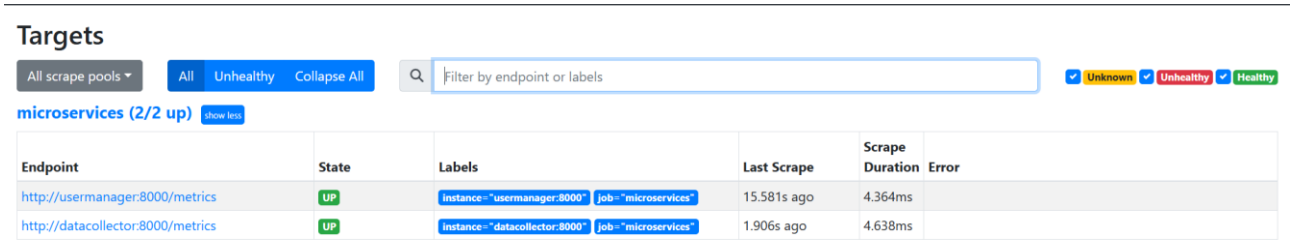
Condizione rilevata: SUPERATA_SOGLIA_BASSA.

--
Questa email è stata esaminata alla ricerca di virus dal software antivirus Avast.
www.avast.com

## 7.4 Monitoraggio delle metriche con Prometheus

L'analisi incrociata degli screenshot di Prometheus mostra un sistema di monitoraggio perfettamente funzionante.

La visualizzazione dello stato dei due microservizi monitorati avviene digitando l'url https://apigate.com/prometheus/targets, mentre la visualizzazione delle metriche monitorate avviene attraverso l'url https://apigate.com/prometheus/graph



**Targets**

All scrape pools | All | Unhealthy | Collapse All | Filter by endpoint or labels | Unknown | Unhealthy | Healthy

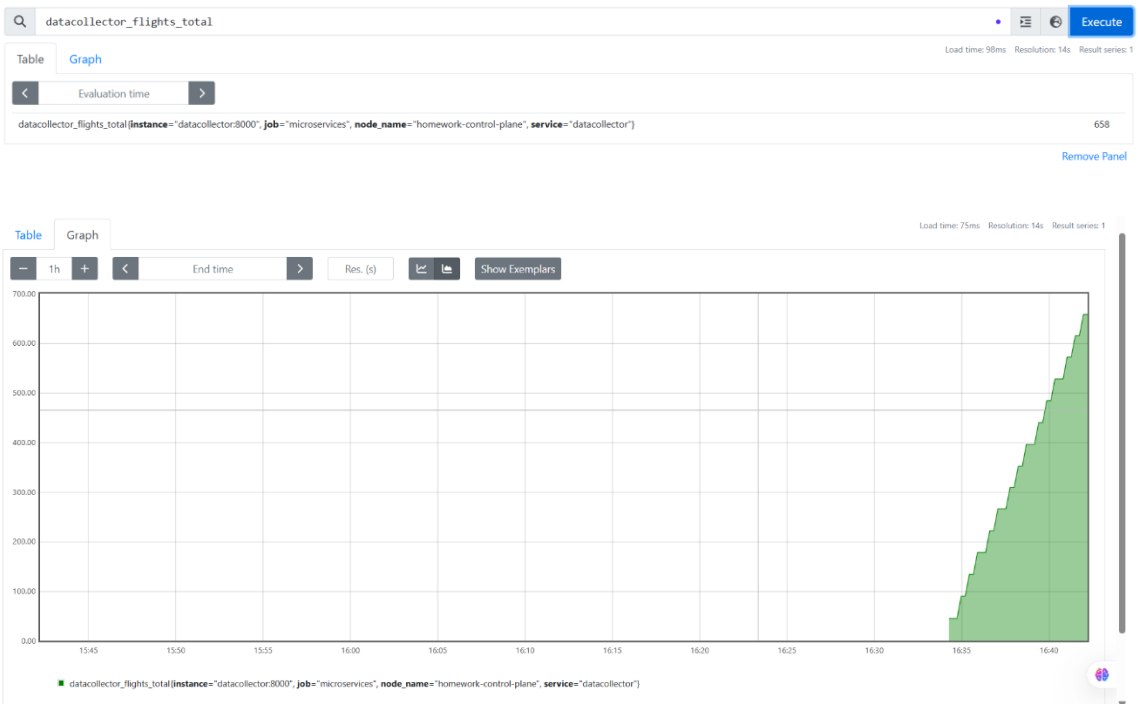
**microservices (2/2 up)** show less

| Endpoint | State | Labels | Last Scrape | Scrape Duration | Error |
|---|---|---|---|---|---|
| http://usermanager:8000/metrics | UP | instance="usermanager:8000" job="microservices" | 15.581s ago | 4.364ms | |
| http://datacollector:8000/metrics | UP | instance="datacollector:8000" job="microservices" | 1.906s ago | 4.638ms | |

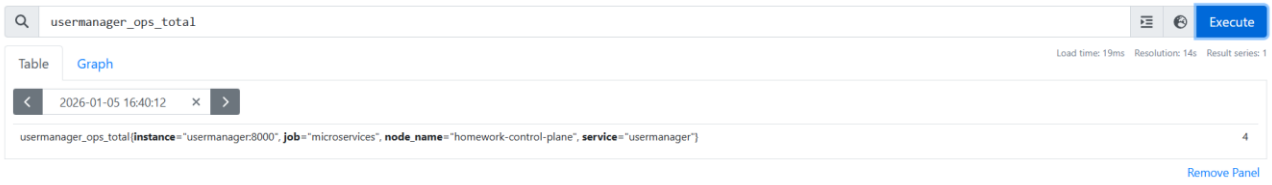
Ecco le osservazioni principali suddivise per tipologia di metrica:

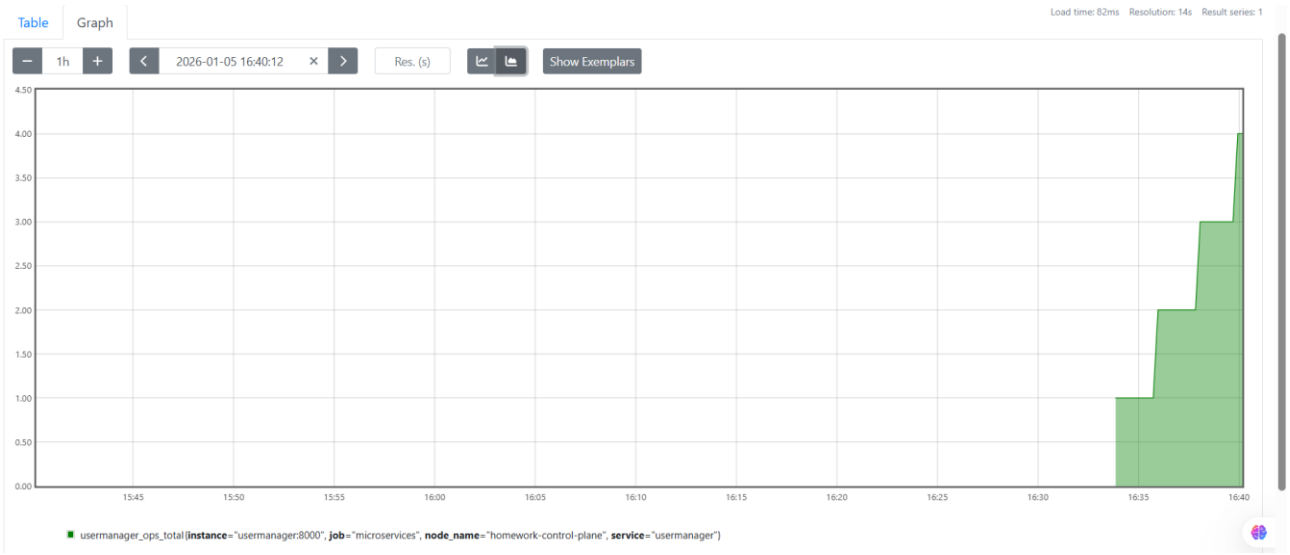
Si analizzi il monitoraggio dei Counter:

- **datacollector_flights_total**: Si osserva una crescita a "gradini" costante, che riflette perfettamente il job ciclico di raccolta dati impostato ogni 20 secondi. Il contatore raggiunge un valore di 658 voli processati, indicando che il microservizio sta raccogliendo dati con successo da OpenSky Network.




- **usermanager_ops_total**: Il grafico mostra incrementi puntuali (linee verticali nel tempo) che corrispondono alle richieste effettive ricevute. Il valore finale di 4 indica che sono state completate con successo quattro operazioni (tre registrazioni e una cancellazioni di utenti) durante la finestra di osservazione.






Si analizzi il monitoraggio dei Gauge:

- **usermanager_db_latency_seconds**: La latenza di scrittura sul database PostgreSQL è estremamente bassa e stabile, con un valore istantaneo di circa 0.011 secondi (11ms). Si notano rari picchi (fino a 1.2s in alcuni grafici), probabilmente dovuti alla fase di inizializzazione o a momenti di saturazione dei volumi durante il commit dei dati.



- **datacollector_grpc_latency_seconds**: Misura il tempo di risposta del canale gRPC verso lo *Usermanager*. Il valore si attesta stabilmente tra 0.01 e 0.03 secondi. È presente un picco isolato molto alto (oltre 0.05s) in un grafico, segno tipico di una latenza di "cold start" della connessione gRPC o di un'interruzione momentanea della rete interna al cluster.




Dalle tabelle di Prometheus si evince che le label obbligatorie sono implementate correttamente:

- **service**: Identifica correttamente l'origine della metrica (usermanager o datacollector).
- **node_name**: Indica il nodo del cluster Kind che ospita il pod, in questo caso homework-control-plane. Questo conferma che il monitoraggio sta leggendo correttamente i dati dall'infrastruttura Kubernetes.
- **instance**: Mostra l'endpoint di scraping sulla porta 8000, come definito nei tuoi manifest app-services.yaml.

## 8. Scelte progettuali

L'applicazione, basata sul paradigma dei sistemi distribuiti, utilizza varie tecnologie allo scopo di rispettare le specifiche assegnate.

Si propone di utilizzare REST API e gRPC per le interazioni sincrone. Nello specifico viene impiegato REST API come interfaccia fra il client, Postman, e il sistema, per permettere agli utenti di utilizzare i servizi messi a disposizione. REST API permette, grazie al formato JSON, una maggiore flessibilità. Inoltre, ogni richiesta REST è stateless, cioè priva di stato, il che permette al sistema di rispondere ad ogni singola richiesta senza dover richiedere ulteriori informazioni. Per la comunicazione interna fra microservizi, come nel caso del DataCollector, che interroga lo UserManager, si è scelto di utilizzare gRPC. La scelta è motivata dal fatto che, per la comunicazione fra microservizi, a differenza di REST, che permette di scambiare solo dati nel formato JSON, gRPC permette, grazie ai Protobuf, la serializzazione dei dati, risultando più efficiente, riducendo la banda utilizzata e velocizzando le comunicazioni.

Per quanto riguarda le interazioni asincrone si è scelto di impiegare Apache Kafka. Questo sistema di messaggistica permette ai microservizi di comunicare senza che ognuno sappia dell'esistenza dell'altro. Ciò permette un alto disaccoppiamento. Se, per esempio, il microservizio AlertNotifierSystem dovesse cadere, i messaggi a lui destinati non vengono persi, ma memorizzati. I microservizi si limitano solamente a pubblicare messaggi nei topic in cui altri microservizi si occuperanno di consumare quei messaggi. Kafka permette: l'ordinamento corretto dei messaggi, in modo da ricevere le notifiche di soglia in maniera coerente ai dati raccolti; garanzia dell' invio grazie al commit dei messaggi, che avviene solo dopo che il messaggio è stato processato (es. commit dopo aver inviato l'email di notifica); persistenza e durability dei messaggi anche quando i producer/consumer dovessero cadere.

L'app dispone di un single-entry point implementato con Nginx: il servizio funge da API gateway e Reverse Proxy nascondendo tutta la complessità del sistema e la granularità dei servizi. Nginx permette di disaccoppiare il client dai dettagli implementativi, fornendo un'interfaccia unificata e sicura grazie a SSL. Il microservizio Nginx conosce i nomi dei microservizi in maniera dettagliata e grazie alle funzioni di reverse proxy renderizza il client al microservizio corretto, anche se quest'ultimo dovesse cambiare posizione fisica (IP). L'utilizzo di SSL, infine, permette di cifrare/decifrare i dati grazie alla chiave privata contenuta in nginx-selsigned.key e alla

chiave pubblica contenuta nel certificato nginx-selfsigned.crt. Le richieste in arrivo alla porta 443, grazie al protocollo TLS, vengono quindi cifrate grazie ad una chiave di sessione scambiata fra le parti usando una cifratura asimmetrica, garantendo riservatezza alle comunicazioni.

Per garantire la politica At-Most-Once, è stato implementato un meccanismo di idempotenza lato server tramite la tabella idempotency_keys, grazie ad una chiave (l'hash dell'email) conservata per un tempo limitato nel database del sistema. Ogni richiesta di registrazione viene tracciata univocamente: se il sistema riceve un duplicato dovuto a un retry di rete, restituisce la risposta salvata senza rieseguire l'operazione. Questa scelta previene inconsistenze e record duplicati nel DB, assicurando che l'azione avvenga al massimo una volta, nonostante l'inaffidabilità della rete. La conservazione per un tempo limitato è gestita dal thread di cleanup (delete_idempotency_key), garantendo che il DB non cresca all'infinito.

Per proteggere il sistema da malfunzionamenti dell'API esterna è stato implementato un circuit-breaker nel DataCollector. In particolare, è stato deciso di creare un'istanza iniziale del circuit breaker, responsabile dell'ottenimento del token di accesso al servizio esterno. Successivamente, è stata creata una mappa di circuit breaker, dove ogni elemento possiede come chiave l'aeroporto di interesse. In questo modo, nel data_collection_job(), le richieste vengono effettuate da circuit breaker diversi, uno per ogni aeroporto. Tale scelta implementativa è stata effettuata tenendo conto sia dei vantaggi, che dei possibili svantaggi. Dipende dal tipo di risposta al fallimento della richiesta. Questa scelta implementativa, da un lato evita che i fallimenti per un aeroporto non vadano ad inficiare sulle richieste di un altro aeroporto per il quale potrebbero essere presenti dei voli, e per ogni aeroporto si ha un circuit breaker che lavora in maniera indipendente rispetto agli altri. Quindi è vantaggioso nei casi di errore 404 da parte del servizio esterno.

Tuttavia, l'implementazione di un singolo circuit breaker sarebbe stata più opportuna considerando errori di tipo 429, in cui terminano le richieste da parte dell'utente, o casi di malfunzionamenti da parte del servizio. In questo modo, anziché aspettare che ogni aeroporto superi la soglia in maniera indipendente, basterebbe superare la soglia considerando i fallimenti di tutti gli aeroporti (fino al raggiungimento della soglia).

Ma, in conclusione, dato l'obiettivo del progetto prefissato, ovvero riuscire a raccogliere periodicamente voli di interesse, se disponibili, ed implementare il pattern circuit breaker in modo che possa attivarsi dopo il raggiungimento di una soglia, la scelta implementativa adottata è risultata la più efficiente. E' importante sottolineare come, grazie all'implementazione del circuit breaker, si evita lo spreco di crediti del proprio account OpenSky.

# 9. Indice delle figure

Figura 1: Schema Architetturale ..... 2
Figura 2: add_user ..... 13
Figura 3: delete_user ..... 14
Figura 4: hard_delete_user ..... 15
Figura 5: add_values ..... 15
Figura 6: data_collection_job ..... 16
Figura 7: process_and_produce ..... 17
Figura 8: publish_flight_count ..... 18
Figura 9: send_notification ..... 19